

Sprawozdanie techniczne z projektu bazy danych

System wypożyczalni samochodów Harvest Motion

Autorzy: Michał Kościanek, Michał Mąka
Przedmiot: Bazy danych
Temat projektu: System obsługi wypożyczalni samochodów

1. Cel projektu

Celem projektu było zaprojektowanie i zaimplementowanie systemu informatycznego wspierającego działanie wypożyczalni samochodów. Aplikacja pozwala na zarządzanie flotą pojazdów, użytkownikami, oddziałami, rezerwacjami, wypożyczeniami, płatnościami, historią serwisową oraz zgłoszeniami uszkodzeń.

Najważniejszym elementem projektu jest relacyjna baza danych PostgreSQL, która przechowuje dane systemowe oraz odpowiada za ich spójność. W projekcie wykorzystano mechanizmy takie jak klucze główne i obce, ograniczenia `CHECK`, `UNIQUE`, typy `ENUM`, indeksy, widoki oraz ograniczenie `EXCLUDE` blokujące nakładanie się aktywnych rezerwacji tego samego samochodu.

System został zrealizowany jako aplikacja webowa składająca się z trzech głównych warstw:

- frontend React, czyli interfejs użytkownika,
- backend Node.js/Express, czyli REST API obsługujące logikę biznesową,
- baza danych PostgreSQL, czyli warstwa trwałego przechowywania danych.

2. Wykorzystane technologie

W projekcie wykorzystano następujące technologie:

Technologia	Zastosowanie
PostgreSQL 15	Relacyjna baza danych
Node.js	Środowisko uruchomieniowe backendu
Express	Framework HTTP do budowy REST API
TypeScript	Typowanie kodu backendu i frontendu
React	Budowa interfejsu użytkownika
Vite	Uruchamianie i budowanie frontendu
Tailwind CSS	Stylowanie aplikacji
Docker Compose	Uruchamianie bazy danych i backendu w kontenerach
pg	Biblioteka backendowa do komunikacji z PostgreSQL

Technologia	Zastosowanie
zod	Walidacja danych wejściowych w API
jsonwebtoken	Obsługa tokenów JWT
bcryptjs	Hashowanie haseł użytkowników

Backend udostępnia REST API na porcie 3000, natomiast baza PostgreSQL działa w kontenerze Docker i jest wystawiona lokalnie na porcie 5433.

3. Ogólna architektura systemu

System został zaprojektowany w architekturze klient-serwer.

Frontend React odpowiada za:

- wyświetlanie widoków aplikacji,
- formularze logowania i rejestracji,
- panel klienta,
- panel pracownika i administratora,
- wysyłanie zapytań HTTP do backendu.

Backend Express odpowiada za:

- obsługę endpointów REST,
- walidację danych wejściowych,
- autoryzację użytkowników,
- wykonywanie zapytań SQL,
- obsługę operacji biznesowych,
- obsługę błędów.

Baza danych PostgreSQL odpowiada za:

- przechowywanie danych,
- wymuszanie integralności relacyjnej,
- kontrolę poprawności danych,
- realizację widoków raportowych,
- blokowanie konfliktów rezerwacji.

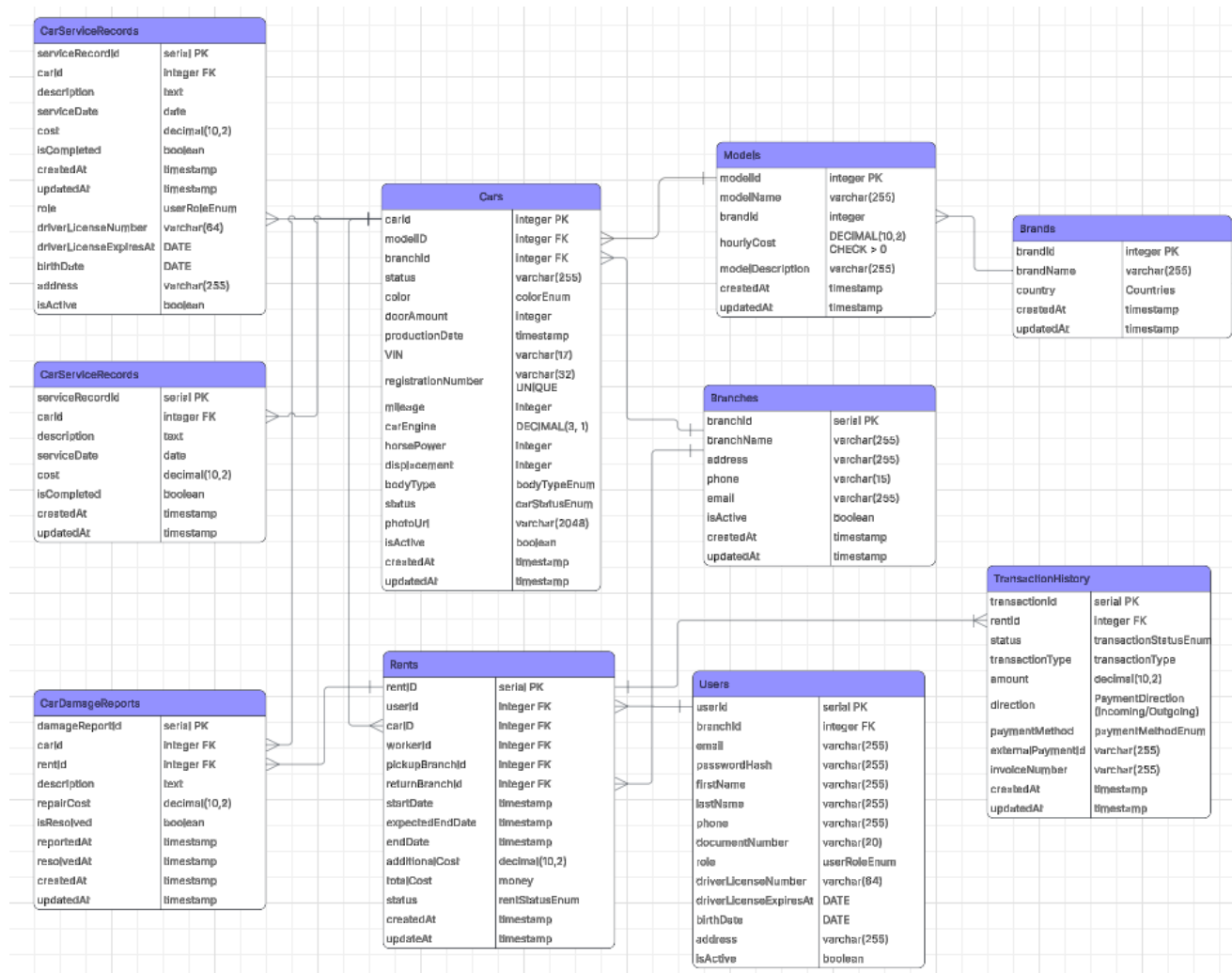
Schemat przepływu danych w systemie:

1. Użytkownik wykonuje akcję w interfejsie React, np. loguje się, wyszukuje auto albo tworzy rezerwację.
2. Frontend wysyła zapytanie HTTP do backendu Express.
3. Backend waliduje dane wejściowe, sprawdza token JWT i uprawnienia użytkownika.
4. Warstwa serwisów backendu wykonuje zapytania SQL do bazy PostgreSQL przez bibliotekę pg.
5. PostgreSQL zwraca wynik zapytania lub błąd naruszenia ograniczeń.
6. Backend mapuje wynik na odpowiedź JSON i odsyła ją do frontendu.

4. Model bazy danych

Baza danych została zaprojektowana jako relacyjny model opisujący wypożyczalnię samochodów. Główne encje to użytkownicy, samochody, modele, marki, oddziały, wypożyczenia, transakcje, serwisy i zgłoszenia uszkodzeń.

Poniżej znajduje się diagram/model bazy danych przygotowany dla projektu:



Najważniejsze relacje widoczne w modelu:

- **Branches** jest powiązane z **Cars** oraz **Users**, ponieważ oddział może posiadać samochody i pracowników,
- **Brands** jest powiązane z **Models**, ponieważ jedna marka może mieć wiele modeli,
- **Models** jest powiązane z **Cars**, ponieważ jeden model może występować w wielu egzemplarzach,
- **Users** jest powiązane z **Rents**, ponieważ klient tworzy wypożyczenia, a pracownik może je obsługiwać,
- **Cars** jest powiązane z **Rents**, ponieważ samochód może być wypożyczany wiele razy,
- **Rents** jest powiązane z **TransactionHistory**, ponieważ wypożyczenie może mieć wiele transakcji,
- **Cars** jest powiązane z **CarServiceRecords** i **CarDamageReports**, ponieważ dla auta można prowadzić historię serwisów i uszkodzeń.

5. Typy ENUM w bazie danych

W projekcie wykorzystano typy **ENUM**, które ograniczają wartości wybranych pól do z góry określonych opcji. Dzięki temu dane są bardziej spójne i trudniej zapisać niepoprawny status lub rolę użytkownika.

Najważniejsze typy:

Typ	Przykładowe wartości	Zastosowanie
Roles	Admin, Manager, Worker, Customer, Banned	Role użytkowników
CarStatus	Available, Rented, Maintenance, Damaged	Status samochodu
RentStatus	Started, Pending, Ended, Cancelled	Status wypożyczenia
TransactionStatus	Rejected, Pending, Accepted	Status płatności
TransactionType	RentalPayment, Deposit, Refund, Penalty, Correction	Typ transakcji
CarColor	White, Black, Silver, Red, Other	Kolor samochodu
BodyTypes	SEDAN, SUV, COUPE, VAN itd.	Typ nadwozia
PaymentMethods	Cash, Card, Bank transfer itd.	Metoda płatności
PaymentDirection	Incoming, Outgoing	Kierunek przepływu środków

Zastosowanie typów **ENUM** powoduje, że baza odrzuca wartości spoza dozwolonego zbioru, np. nie można zapisać statusu samochodu jako **Free**, jeżeli dopuszczalna wartość to **Available**.

6. Tabele bazy danych

6.1. Tabela **Branches**

Tabela **Branches** przechowuje informacje o oddziałach wypożyczalni.

Najważniejsze kolumny:

Kolumna	Typ	Opis
branchId	SERIAL PRIMARY KEY	Identyfikator oddziału
branchName	VARCHAR(255)	Nazwa oddziału
address	VARCHAR(255)	Adres oddziału
phone	VARCHAR(15)	Numer telefonu
email	VARCHAR(255)	Email oddziału
isActive	BOOLEAN	Informacja, czy oddział jest aktywny
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **branchName** jest unikalny,
- **phone** musi pasować do wzorca numeru telefonu,
- **email** musi pasować do wzorca adresu email,
- **isActive** ma domyślną wartość **TRUE**.

Tabela jest wykorzystywana do przypisywania samochodów i pracowników do konkretnych lokalizacji.

6.2. Tabela **Brands**

Tabela **Brands** przechowuje marki samochodów.

Kolumna	Typ	Opis
brandId	SERIAL PRIMARY KEY	Identyfikator marki
brandName	VARCHAR(255)	Nazwa marki
country	Countries	Kraj pochodzenia marki
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **brandName** jest unikalny,
- **country** jest typem **ENUM**, więc musi przyjmować jedną z dopuszczalnych wartości.

Tabela **Brands** jest powiązana relacją jeden-do-wielu z tabelą **Models**.

6.3. Tabela **Models**

Tabela **Models** opisuje modele samochodów.

Kolumna	Typ	Opis
modelId	SERIAL PRIMARY KEY	Identyfikator modelu
modelName	VARCHAR(255)	Nazwa modelu
brandId	INTEGER	Klucz obcy do tabeli Brands
hourlyCost	DECIMAL(10,2)	Koszt wypożyczenia za godzinę
modelDescription	VARCHAR(255)	Opis modelu
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **brandId** wskazuje na istniejącą markę,
- **hourlyCost** musi być większy od zera,
- para (**brandId**, **modelName**) jest unikalna.

Oddzielenie marek i modeli od konkretnych samochodów jest przykładem normalizacji danych. Dzięki temu nie trzeba powtarzać informacji o marce i modelu przy każdym egzemplarzu pojazdu.

6.4. Tabela **Cars**

Tabela **Cars** przechowuje konkretne egzemplarze samochodów dostępne w wypożyczalni.

Kolumna	Typ	Opis
carId	SERIAL PRIMARY KEY	Identyfikator samochodu
modelId	INTEGER	Klucz obcy do tabeli Models
branchId	INTEGER	Klucz obcy do tabeli Branches
status	CarStatus	Status samochodu
color	CarColor	Kolor samochodu
doorAmount	INTEGER	Liczba drzwi
productionDate	DATE	Data produkcji
VIN	VARCHAR(17)	Numer VIN
registrationNumber	VARCHAR(32)	Numer rejestracyjny
mileage	INTEGER	Przebieg
carEngine	DECIMAL(3,1)	Pojemność silnika
horsePower	INTEGER	Moc pojazdu
bodyType	BodyTypes	Typ nadwozia
imageUrl	VARCHAR(2048)	Link do zdjęcia samochodu
isActive	BOOLEAN	Czy rekord jest aktywny
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **VIN** jest unikalny i musi mieć poprawny format,
- **registrationNumber** jest unikalny,
- **doorAmount** musi być większe od zera,
- **productionDate** nie może być z przyszłości,
- **mileage** nie może być ujemny,
- **horsePower** musi być większe od zera,
- **modelId** wskazuje na istniejący model,
- **branchId** wskazuje na istniejący oddział lub może być **NULL**.

Tabela **Cars** jest centralną tabelą systemu, ponieważ łączy flotę z oddziałami, wypożyczeniami, serwisami i zgłoszeniami uszkodzeń.

6.5. Tabela **Users**

Tabela **Users** przechowuje dane wszystkich użytkowników systemu: klientów, pracowników, managerów i administratorów.

Kolumna	Typ	Opis
<code>userId</code>	<code>SERIAL PRIMARY KEY</code>	Identyfikator użytkownika
<code>email</code>	<code>VARCHAR(255)</code>	Email użytkownika
<code>passwordHash</code>	<code>VARCHAR(255)</code>	Zahashowane hasło
<code>firstName</code>	<code>VARCHAR(255)</code>	Imię
<code>lastName</code>	<code>VARCHAR(255)</code>	Nazwisko
<code>phone</code>	<code>VARCHAR(15)</code>	Numer telefonu
<code>role</code>	<code>Roles</code>	Rola użytkownika
<code>branchId</code>	<code>INTEGER</code>	Przypisany oddział dla pracowników
<code>driverLicenseNumber</code>	<code>VARCHAR(64)</code>	Numer prawa jazdy
<code>driverLicenseExpiresAt</code>	<code>DATE</code>	Data ważności prawa jazdy
<code>birthDate</code>	<code>DATE</code>	Data urodzenia
<code>address</code>	<code>VARCHAR(255)</code>	Adres użytkownika
<code>isActive</code>	<code>BOOLEAN</code>	Czy konto jest aktywne
<code>createdAt, updatedAt</code>	<code>TIMESTAMP</code>	Daty utworzenia i aktualizacji

Ograniczenia:

- `email` jest unikalny i musi mieć poprawny format,
- `phone` jest unikalny i musi mieć poprawny format,
- `role` musi być jedną z wartości typu `Roles`,
- `birthDate` nie może być datą z przyszłości,
- `branchId` wskazuje na oddział.

Hasła nie są przechowywane w postaci jawnej. Backend hashuje je przy pomocy biblioteki `bcryptjs`.

6.6. Tabela **Rents**

Tabela **Rents** przechowuje rezerwacje i wypożyczenia samochodów.

Kolumna	Typ	Opis
<code>rentId</code>	<code>SERIAL PRIMARY KEY</code>	Identyfikator wypożyczenia

Kolumna	Typ	Opis
userId	INTEGER	Klient tworzący wypożyczenie
carId	INTEGER	Wypożyczany samochód
workerId	INTEGER	Pracownik obsługujący wypożyczenie
pickupBranchId	INTEGER	Oddział odbioru
returnBranchId	INTEGER	Oddział zwrotu
startDate	TIMESTAMP	Data rozpoczęcia
expectedEndDate	TIMESTAMP	Planowana data zakończenia
endDate	TIMESTAMP	Faktyczna data zakończenia
additionalCost	DECIMAL(10,2)	Koszty dodatkowe
totalCost	DECIMAL(10,2)	Koszt całkowity
status	RentStatus	Status wypożyczenia
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **userId** musi wskazywać istniejącego użytkownika,
- **carId** musi wskazywać istniejący samochód,
- **expectedEndDate** musi być późniejsze niż **startDate**,
- **endDate** nie może być wcześniejsze niż **startDate**,
- **additionalCost** nie może być ujemny,
- **totalCost** nie może być ujemny,
- status i daty muszą być spójne.

Najważniejsze ograniczenie w tabeli **Rents**:

```
CONSTRAINT ex_rents_no_car_overlap EXCLUDE USING gist (
    carId WITH =,
    tsrange(startDate, COALESCE(endDate, expectedEndDate), '[]') WITH &&
) WHERE (status IN ('Pending', 'Started'))
```

Ograniczenie to blokuje utworzenie dwóch aktywnych rezerwacji tego samego samochodu w nakładającym się przedziale czasu. Jest to bardzo ważne, ponieważ część logiki biznesowej jest zabezpieczona bezpośrednio na poziomie bazy danych, a nie wyłącznie w kodzie aplikacji.

6.7. Tabela **TransactionHistory**

Tabela **TransactionHistory** przechowuje historię płatności i innych operacji finansowych.

Kolumna	Typ	Opis
transactionId	SERIAL PRIMARY KEY	Identyfikator transakcji
rentId	INTEGER	Powiązane wypożyczenie
status	TransactionStatus	Status transakcji
transactionType	TransactionType	Typ transakcji
amount	DECIMAL(10,2)	Kwota
direction	PaymentDirection	Przychód lub wydatek
paymentMethod	PaymentMethods	Metoda płatności
externalPaymentId	VARCHAR(255)	Identyfikator zewnętrzny płatności
invoiceNumber	VARCHAR(255)	Numer faktury
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **rentId** wskazuje na istniejące wypożyczenie,
- **amount** musi być większe od zera,
- zaakceptowana transakcja musi mieć podaną metodę płatności.

Tabela ta pozwala przechowywać zarówno płatności za wynajem, jak i kary, zwroty, depozyty oraz korekty.

6.8. Tabela **CarServiceRecords**

Tabela **CarServiceRecords** przechowuje historię serwisową pojazdów.

Kolumna	Typ	Opis
serviceRecordId	SERIAL PRIMARY KEY	Identyfikator wpisu serwisowego
carId	INTEGER	Samochód, którego dotyczy wpis
description	TEXT	Opis wykonanej lub planowanej usługi
serviceDate	DATE	Data serwisu
cost	DECIMAL(10,2)	Koszt serwisu
isCompleted	BOOLEAN	Czy serwis został zakończony
createdAt, updatedAt	TIMESTAMP	Daty utworzenia i aktualizacji

Ograniczenia:

- **carId** wskazuje na istniejący samochód,
- **description** jest wymagane,
- **cost** nie może być ujemny.

Tabela umożliwia kontrolę kosztów utrzymania floty.

6.9. Tabela `CarDamageReports`

Tabela `CarDamageReports` przechowuje zgłoszenia uszkodzeń samochodów.

Kolumna	Typ	Opis
<code>damageReportId</code>	<code>SERIAL PRIMARY KEY</code>	Identyfikator zgłoszenia
<code>carId</code>	<code>INTEGER</code>	Samochód, którego dotyczy zgłoszenie
<code>rentId</code>	<code>INTEGER</code>	Wypożyczenie powiązane ze szkodą
<code>description</code>	<code>TEXT</code>	Opis uszkodzenia
<code>repairCost</code>	<code>DECIMAL(10,2)</code>	Koszt naprawy
<code>isResolved</code>	<code>BOOLEAN</code>	Czy zgłoszenie rozwiązano
<code>reportedAt</code>	<code>TIMESTAMP</code>	Data zgłoszenia
<code>resolvedAt</code>	<code>TIMESTAMP</code>	Data rozwiązania
<code>createdAt, updatedAt</code>	<code>TIMESTAMP</code>	Daty utworzenia i aktualizacji

Ograniczenia:

- `carId` wskazuje na istniejący samochód,
- `rentId` może wskazywać na wypożyczenie, ale może też być `NULL`,
- `repairCost` nie może być ujemny,
- jeżeli `isResolved = FALSE`, to `resolvedAt` musi być `NULL`,
- jeżeli `isResolved = TRUE`, to `resolvedAt` musi być ustawione.

Tabela pozwala powiązać szkodę z konkretnym pojazdem i opcjonalnie z konkretnym wypożyczeniem.

7. Indeksy

W bazie utworzono indeksy na kolumnach często wykorzystywanych w filtrowaniu i łączeniu tabel.

Przykłady indeksów:

Indeks	Tabela	Zastosowanie
<code>idx_models_brand_id</code>	<code>Models</code>	Szybsze pobieranie modeli danej marki
<code>idx_cars_model_id</code>	<code>Cars</code>	Szybsze łączenie samochodów z modelami
<code>idx_cars_status</code>	<code>Cars</code>	Szybsze filtrowanie po statusie
<code>idx_cars_branch_id</code>	<code>Cars</code>	Szybsze pobieranie aut z oddziału
<code>idx_users_branch_id</code>	<code>Users</code>	Szybsze pobieranie pracowników oddziału

Indeks	Tabela	Zastosowanie
<code>idx_rents_user_id</code>	<code>Rents</code>	Szybsze pobieranie wypożyczeń użytkownika
<code>idx_rents_car_id</code>	<code>Rents</code>	Szybsze pobieranie wypożyczeń samochodu
<code>idx_rents_status</code>	<code>Rents</code>	Szybsze filtrowanie wypożyczeń po statusie
<code>idx_rents_dates</code>	<code>Rents</code>	Szybsza obsługa zapytań po datach
<code>idx_transactions_rent_id</code>	<code>TransactionHistory</code>	Szybsze pobieranie transakcji dla wypożyczenia

Indeksy poprawiają wydajność odczytu danych, szczególnie w miejscach, gdzie backend często filtruje po `status`, `userId`, `carId`, `branchId` lub datach wypożyczeń.

8. Widoki w bazie danych

Projekt wykorzystuje widoki SQL do przygotowania danych raportowych i często używanych zestawień.

8.1. `vw_current_rentals_by_user`

Widok zwraca aktualnie trwające wypożyczenia. Łączy dane z tabel `Rents`, `Users`, `Cars`, `Models` i `Brands`.

Warunek:

```
WHERE r.status = 'Started'
AND r.endDate IS NULL
```

Widok jest wykorzystywany do raportów aktualnych wypożyczeń.

8.2. `vw_all_rentals_by_user`

Widok zwraca pełną historię wypożyczeń użytkowników. Jest używany np. przy raporcie historii klienta.

8.3. `vw_available_cars`

Widok zwraca samochody dostępne do wypożyczenia.

Warunek:

```
WHERE c.status = 'Available'
AND c.isActive = TRUE
```

Widok upraszcza pobieranie listy aut dostępnych dla klientów.

8.4. `vw_most_popular_cars`

Widok zlicza liczbę wypożyczeń dla modeli samochodów.

Wykorzystywany jest do raportu popularności samochodów.

8.5. vw_overdue_rentals

Widok zwraca wypożyczenia, które są aktywne i przekroczyły planowaną datę zakończenia.

Warunek:

```
WHERE r.status = 'Started'
      AND r.endDate IS NULL
      AND r.expectedEndDate < NOW()
```

8.6. vw_revenue_by_day

Widok agreguje przychody i wydatki według dnia. Jest przydatny przy raportowaniu finansowym.

9. Funkcjonalności systemu

System obsługuje kilka typów użytkowników. Dostęp do funkcji zależy od roli zapisanej w tabeli **Users**.

9.1. Funkcjonalności klienta

Klient może:

- zarejestrować konto,
- zalogować się,
- przeglądać dostępne samochody,
- filtrować flotę,
- utworzyć rezerwację,
- sprawdzić swoje aktualne wypożyczenia,
- sprawdzić historię swoich wynajmów.

Najważniejsze endpointy klienta:

Endpoint	Opis
POST /auth/register	Rejestracja klienta
POST /auth/login	Logowanie
GET /cars	Lista samochodów
GET /cars/available	Lista dostępnych aut
GET /cars/popular	Lista popularnych samochodów na stronie głównej
GET /rents/my	Wypożyczenia zalogowanego klienta

Endpoint	Opis
POST /rents	Utworzenie rezerwacji
GET /rents/availability	Sprawdzenie dostępności samochodu

9.2. Funkcjonalności pracownika

Pracownik może:

- przeglądać wypożyczenia,
- rozpoczynać wypożyczenia,
- kończyć wypożyczenia,
- anulować rezerwacje typu no-show,
- aktualizować status samochodu,
- rejestrować serwis samochodu,
- zgłaszać uszkodzenia,
- obsługiwać transakcje.

Najważniejsze endpointy:

Endpoint	Opis
GET /rents	Lista wypożyczeń
GET /rents/pending	Rezerwacje oczekujące
POST /rents/:id/start	Rozpoczęcie wypożyczenia
POST /rents/:id/finish	Zakończenie wypożyczenia
POST /rents/:id/cancel-no-show	Anulowanie nieodebranej rezerwacji
PATCH /cars/:id/status	Zmiana statusu samochodu
POST /car-service-records	Dodanie wpisu serwisowego
POST /car-damage-reports	Dodanie zgłoszenia uszkodzenia
POST /transactions	Dodanie transakcji

9.3. Funkcjonalności managera

Manager może:

- zarządzać markami,
- zarządzać modelami,
- zarządzać samochodami,
- przeglądać raporty,
- obsługiwać część danych pracowniczych i operacyjnych.

Najważniejsze endpointy:

Endpoint	Opis
POST /brands	Dodanie marki
PATCH /brands/:id	Aktualizacja marki
POST /models	Dodanie modelu
PATCH /models/:id	Aktualizacja modelu
POST /cars	Dodanie samochodu
PATCH /cars/:id	Aktualizacja samochodu
POST /cars/upload-image	Wgranie zdjęcia samochodu
GET /reports/current-rentals	Raport aktualnych wypożyczeń
GET /reports/popular-cars	Raport popularnych aut
GET /reports/overdue-rents	Raport przeterminowanych wypożyczeń
GET /reports/revenue	Raport przychodów

9.4. Funkcjonalności administratora

Administrator ma najszersze uprawnienia. Może:

- zarządzać użytkownikami,
- zmieniać role użytkowników,
- tworzyć pracowników i managerów,
- zarządzać oddziałami,
- zarządzać flotą,
- korzystać z raportów.

Najważniejsze endpointy:

Endpoint	Opis
GET /users	Lista użytkowników
POST /users	Utworzenie użytkownika
PATCH /users/:id	Aktualizacja użytkownika
PATCH /users/:id/role	Zmiana roli
PATCH /users/:id/password	Zmiana hasła
DELETE /users/:id	Usunięcie lub dezaktywacja użytkownika
GET /branches	Lista oddziałów
GET /branches/:id/cars	Samochody w oddziale
POST /branches	Dodanie oddziału

Endpoint	Opis
PATCH /branches/:id	Aktualizacja oddziału
DELETE /branches/:id	Usunięcie oddziału

9.5. Funkcjonalności interfejsu użytkownika

Frontend projektu zawiera kilka widoków odpowiadających głównym procesom systemu:

Widok	Funkcjonalność
Strona główna	Prezentacja popularnych samochodów i przejścia do wyszukiwania
Logowanie	Logowanie zwykłego użytkownika i konto testowe administratora
Rejestracja	Utworzenie konta klienta
Wyszukiwarka samochodów	Przeglądanie i filtrowanie floty
Tworzenie rezerwacji	Wybór samochodu oraz terminu wypożyczenia
Panel klienta	Podgląd aktywnego wynajmu, rezerwacji i ostatnich podróży
Historia klienta	Lista wypożyczeń klienta z filtrowaniem po statusie
Panel pracownika	Obsługa wypożyczeń, start, zakończenie i anulowanie no-show
Zarządzanie flotą	Dodawanie i edycja aut, zmiana statusu, zdjęcia samochodów

Po audycie tras API i widoków frontendu można stwierdzić, że sprawozdanie obejmuje wszystkie główne moduły projektu: autoryzację, użytkowników, oddziały, marki, modele, samochody, rezerwacje/wypożyczenia, transakcje, raporty, serwisy oraz zgłoszenia uszkodzeń. Projekt nie posiada osobnego endpointu **DELETE** dla transakcji, co jest celowe, ponieważ historia finansowa powinna być zachowywana.

10. Autoryzacja i bezpieczeństwo

Logowanie użytkownika odbywa się przez endpoint:

```
POST /auth/login
```

Backend sprawdza email i hasło. Hasła są porównywane z haszem zapisanym w bazie przy użyciu **bcryptjs**.

Po poprawnym logowaniu backend zwraca token JWT. Token zawiera:

- identyfikator użytkownika,
- email,
- rolę,

- identyfikator oddziału.

Dostęp do chronionych endpointów wymaga nagłówka:

```
Authorization: Bearer <token>
```

System posiada middleware:

- **authenticate** - sprawdza poprawność tokena,
- **requireRoles** - ogranicza dostęp do wybranych ról,
- **requireSelfOrRoles** - pozwala użytkownikowi działać na własnych danych albo wybranym rolaom działać administracyjnie.

Przykład:

```
router.post(
  '/',
  authenticate,
  requireRoles('Manager', 'Admin'),
  validate({ body: carCreateSchema }),
  asyncHandler(carsController.createCar),
);
```

Oznacza to, że samochód może dodać tylko zalogowany użytkownik z rolą **Manager** albo **Admin**.

11. Walidacja danych wejściowych

Backend wykorzystuje bibliotekę **zod** do walidacji danych wejściowych. Walidacja wykonywana jest przed uruchomieniem kontrolera i zapytania SQL.

Przykłady walidacji:

- email musi mieć poprawny format,
- hasło musi mieć minimalną długość,
- identyfikator musi być liczbą dodatnią,
- status samochodu musi być jednym z dozwolonych statusów,
- data zakończenia wypożyczenia musi być poprawną datą,
- koszt musi być liczbą nieujemną lub dodatnią.

Walidacja występuje zarówno na poziomie backendu, jak i bazy danych. Dzięki temu system jest zabezpieczony podwójnie:

- aplikacja odrzuca niepoprawne dane wcześniej,
- baza danych nadal pilnuje integralności, jeżeli dane trafiłyby do niej z innego źródła.

12. Operacje CRUD w projekcie

CRUD oznacza cztery podstawowe operacje na danych:

- **Create** - tworzenie rekordu,
- **Read** - odczyt rekordu lub listy rekordów,
- **Update** - aktualizacja rekordu,
- **Delete** - usuwanie rekordu.

W projekcie operacje CRUD są realizowane przez REST API. Każdy moduł ma osobny plik tras, kontroler i serwis. Kontroler obsługuje żądanie HTTP, a serwis wykonuje właściwe zapytania SQL.

Ogólny schemat działania operacji CRUD:

1. Frontend wysyła żądanie HTTP, np. **GET /cars**, **POST /rents** albo **PATCH /users/:id**.
2. Router Express dopasowuje żądanie do odpowiedniego endpointu.
3. Middleware **zod** waliduje parametry, query string albo body żądania.
4. Middleware autoryzacji sprawdza token JWT i rolę użytkownika, jeżeli endpoint jest chroniony.
5. Kontroler przekazuje dane do odpowiedniej funkcji serwisowej.
6. Serwis wykonuje zapytanie SQL lub transakcję w PostgreSQL.
7. Baza danych zwraca wynik albo błąd ograniczeń.
8. Backend odsyła odpowiedź JSON do frontendu.

13. CRUD dla użytkowników

Moduł użytkowników obsługuje tabelę **Users**.

Create

Utworzenie użytkownika odbywa się przez:

```
POST /users
```

lub w przypadku klienta:

```
POST /auth/register
```

Podczas tworzenia użytkownika backend:

1. waliduje dane wejściowe,
2. hashuje hasło przy użyciu **bcryptjs**,
3. zapisuje użytkownika do tabeli **Users**,
4. zwraca dane użytkownika bez hasła.

Hasło nie jest nigdy zwracane w odpowiedzi API.

Read

Odczyt użytkowników:

```
GET /users
GET /users/:id
GET /users/:id/rents
```

Lista użytkowników jest dostępna tylko dla ról **Manager** i **Admin**.

Update

Aktualizacja użytkownika:

```
PATCH /users/:id
PATCH /users/:id/role
PATCH /users/:id/password
```

Możliwa jest zmiana danych osobowych, roli oraz hasła. Zmiana hasła powoduje ponowne zapisanie go jako hash.

Delete

Usunięcie użytkownika:

```
DELETE /users/:id
```

Jeżeli użytkownik ma powiązane wypożyczenia, system nie usuwa go fizycznie, tylko dezaktywuje konto i ustawia rolę **Banned**. Jest to przykład tzw. miękkiego usuwania w sytuacji, gdy rekord jest powiązany z historią biznesową.

14. CRUD dla oddziałów

Oddziały wypożyczalni są obsługiwane przez tabelę **Branches**. Moduł ten pozwala zarządzać lokalizacjami, w których znajdują się samochody i do których mogą być przypisani pracownicy.

Create

```
POST /branches
```

Oddział może utworzyć użytkownik z rolą **Admin**. Przy tworzeniu oddziału podawane są nazwa, adres, telefon, email oraz informacja, czy oddział jest aktywny. Backend waliduje format telefonu i adresu email, a

baza dodatkowo wymusza unikalność nazwy oddziału.

Read

```
GET /branches
GET /branches/:id
GET /branches/:id/cars
```

System pozwala pobrać listę oddziałów, szczegóły pojedynczego oddziału oraz samochody przypisane do wybranego oddziału. Odczyt oddziałów jest wykorzystywany m.in. przy przypisywaniu aut do lokalizacji oraz przy obsłudze procesu wypożyczeń.

Update

```
PATCH /branches/:id
```

Aktualizacja oddziału pozwala zmienić nazwę, adres, telefon, email oraz status aktywności. Operacja jest dostępna dla ról **Admin** i **Manager**.

Delete

```
DELETE /branches/:id
```

Usunięcie oddziału jest dostępne dla roli **Admin**. Jeżeli oddział jest powiązany z samochodami lub użytkownikami, relacje w schemacie ustawiają odpowiednie klucze obce na **NULL** przez **ON DELETE SET NULL**. Dzięki temu usunięcie oddziału nie usuwa automatycznie samochodów ani użytkowników.

15. CRUD dla marek i modeli

Marki są przechowywane w tabeli **Brands**, a modele w tabeli **Models**.

Create

```
POST /brands
POST /models
```

Markę i model może utworzyć **Manager** albo **Admin**.

Read

```
GET /brands
GET /brands/:id
GET /brands/:brandId/models
GET /models
GET /models/:id
GET /models/by-brand/:brandId
```

Odczyt marek i modeli jest wykorzystywany np. w formularzach dodawania samochodu i filtrowania floty.

Update

```
PATCH /brands/:id
PATCH /models/:id
```

Aktualizacja pozwala zmienić nazwę marki, kraj, nazwę modelu, koszt godzinowy i opis modelu.

Delete

```
DELETE /brands/:id
DELETE /models/:id
```

Usuwanie jest ograniczone przez klucze obce. Nie można bezpiecznie usunąć marki lub modelu, jeżeli istnieją powiązane rekordy, np. samochody.

16. CRUD dla samochodów

Samochody są obsługiwane przez tabelę **Cars**.

Create

```
POST /cars
```

Samochód może dodać **Manager** albo **Admin**.

Backend obsługuje dwa warianty dodawania samochodu:

1. dodanie samochodu z istniejącym **modelId**,
2. dodanie samochodu razem z nową marką i nowym modelem.

W drugim wariantcie backend używa transakcji bazodanowej. Najpierw sprawdza lub tworzy markę, potem sprawdza lub tworzy model, a dopiero na końcu dodaje samochód. Dzięki transakcji operacja jest atomowa: albo wszystkie kroki się wykonają, albo żaden.

Read

```
GET /cars
GET /cars/available
GET /cars/popular
GET /cars/:id
GET /cars/:id/rents
```

Lista samochodów może być filtrowana po:

- marce,
- modelu,
- oddziale,
- statusie,
- kolorze,
- typie nadwozia,
- minimalnym i maksymalnym koszcie,
- dostępności w konkretnym terminie.

Przy filtrowaniu po terminie backend sprawdza, czy nie istnieje aktywne wypożyczenie nachodzące na wybrany zakres dat.

Update

```
PATCH /cars/:id
PATCH /cars/:id/status
```

Aktualizacja samochodu pozwala zmienić dane techniczne, przypisanie do oddziału, status, przebieg, numer rejestracyjny lub zdjęcie.

Zmiana statusu jest dostępna dla ról **Worker**, **Manager** i **Admin**.

Delete

```
DELETE /cars/:id
```

Jeżeli samochód ma historię wypożyczeń, system nie usuwa go fizycznie, tylko ustawia **isActive = FALSE**. Dzięki temu nie znika historia wypożyczeń i raporty nadal są spójne.

17. CRUD i proces biznesowy wypożyczeń

Wypożyczenia są najważniejszym procesem biznesowym systemu. Są obsługiwane przez tabelę **Rents**.

Create

```
POST /rents
```

Tworzenie rezerwacji działa następująco:

1. backend sprawdza, czy użytkownik może wypożyczać samochody,
2. sprawdza, czy samochód istnieje,
3. sprawdza, czy samochód jest przypisany do odpowiedniego oddziału,
4. sprawdza dostępność w wybranym terminie,
5. zapisuje rekord w tabeli **Rents**.

Jeżeli klient tworzy rezerwację samodzielnie, backend automatycznie przypisuje **userId** z tokena JWT. Dzięki temu klient nie może utworzyć rezerwacji na konto innego użytkownika.

Read

```
GET /rents
GET /rents/my
GET /rents/current
GET /rents/pending
GET /rents/overdue
GET /rents/availability
GET /rents/:id
```

System pozwala pobierać:

- wszystkie wypożyczenia,
- wypożyczenia zalogowanego klienta,
- aktualne wypożyczenia,
- rezerwacje oczekujące,
- wypożyczenia przeterminowane,
- dostępność samochodu w danym terminie.

Update

```
PATCH /rents/:id
POST /rents/:id/start
POST /rents/:id/finish
POST /rents/:id/cancel-no-show
```

Aktualizacja wypożyczenia ma kilka wariantów:

- edycja danych rezerwacji,

- rozpoczęcie wypożyczenia,
- zakończenie wypożyczenia,
- anulowanie rezerwacji w przypadku nieodebrania auta.

Przy rozpoczęciu wypożyczenia:

1. system sprawdza, czy wypożyczenie ma status **Pending**,
2. sprawdza dostępność samochodu,
3. zmienia status wypożyczenia na **Started**,
4. zmienia status samochodu na **Rented**.

Przy zakończeniu wypożyczenia:

1. system sprawdza, czy wypożyczenie ma status **Started**,
2. oblicza koszt na podstawie czasu i ceny godzinowej,
3. dolicza koszty dodatkowe i ewentualną opłatę za spóźnienie,
4. ustawia status wypożyczenia na **Ended**,
5. aktualizuje status samochodu,
6. opcjonalnie tworzy transakcję płatności,
7. opcjonalnie tworzy zgłoszenie uszkodzenia.

Ta operacja jest wykonywana w transakcji bazodanowej, ponieważ zmienia kilka tabel jednocześnie: **Rents**, **Cars**, **TransactionHistory** i opcjonalnie **CarDamageReports**.

Delete

```
DELETE /rents/:id
```

Usuwanie wypożyczenia jest ograniczone. System pozwala usunąć tylko rezerwację oczekującą bez transakcji. Jeżeli wypożyczenie ma powiązaną historię, zamiast fizycznego usunięcia może zostać anulowane.

18. CRUD dla transakcji

Transakcje są obsługiwane przez tabelę **TransactionHistory**.

Create

```
POST /transactions
```

Tworzenie transakcji wymaga powiązania z istniejącym wypożyczeniem.

Read

```
GET /transactions
GET /transactions/by-rent/:rentId
GET /transactions/:id
```

System pozwala pobrać wszystkie transakcje, transakcje dla konkretnego wypożyczenia oraz pojedynczą transakcję.

Update

```
PATCH /transactions/:id
PATCH /transactions/:id/status
```

Możliwa jest aktualizacja statusu, typu, kwoty, kierunku płatności, metody płatności, identyfikatora zewnętrznego oraz numeru faktury.

Delete

W projekcie nie udostępniono klasycznego endpointu usuwania transakcji. Jest to uzasadnione biznesowo, ponieważ historia płatności powinna być trwała. Zamiast usuwać płatności, system może rejestrować korekty lub zwroty jako nowe transakcje.

19. CRUD dla serwisów samochodów

Historia serwisowa jest obsługiwana przez tabelę `CarServiceRecords`.

Create

```
POST /car-service-records
```

Dodanie wpisu serwisowego wymaga wskazania samochodu, opisu, daty, kosztu oraz informacji, czy usługa została zakończona.

Read

```
GET /car-service-records
GET /car-service-records/by-car/:id
```

System pozwala pobrać wszystkie wpisy serwisowe albo wpisy dla jednego samochodu.

Update


```
PATCH /car-service-records/:id
```

Możliwa jest aktualizacja opisu, daty, kosztu i statusu wykonania.

Delete

```
DELETE /car-service-records/:id
```

Usunięcie wpisu serwisowego usuwa rekord z bazy, jeżeli istnieje.

20. CRUD dla zgłoszeń uszkodzeń

Zgłoszenia uszkodzeń są obsługiwane przez tabelę `CarDamageReports`.

Create

```
POST /car-damage-reports
```

Zgłoszenie zawiera samochód, opcjonalne wypożyczenie, opis uszkodzenia i przewidywany koszt naprawy.

Read

```
GET /car-damage-reports
```

System pozwala pobrać listę zgłoszeń uszkodzeń.

Update

```
PATCH /car-damage-reports/:id
```

Aktualizacja pozwala zmienić opis, koszt naprawy i status rozwiązania. Jeżeli zgłoszenie zostanie oznaczone jako rozwiązane, baza wymaga ustawienia daty rozwiązania.

Delete

```
DELETE /car-damage-reports/:id
```

Usuwanie jest możliwe dla zgłoszeń nierozwiązanych. Pozwala to uniknąć usuwania zamkniętej historii napraw.

21. Raporty

System posiada moduł raportów dostępny dla ról **Manager** i **Admin**.

Endpointy:

Endpoint	Opis
GET /reports/current-rentals	Aktualne wypożyczenia
GET /reports/popular-cars	Najpopularniejsze samochody
GET /reports/overdue-rents	Przeterminowane wypożyczenia
GET /reports/revenue	Przychody
GET /reports/customer-history/:id	Historia klienta

Raporty korzystają głównie z widoków SQL. Dzięki temu część logiki agregowania danych znajduje się w bazie, a backend może pobierać gotowe zestawienia.

Przykład raportu przychodów grupuje transakcje po dacie i oblicza:

- przychody,
- wydatki,
- wynik netto,
- liczbę transakcji.

22. Transakcje bazodanowe

W projekcie wykorzystano transakcje bazodanowe w miejscach, gdzie jedna operacja biznesowa zmienia kilka tabel.

Przykłady:

- dodanie samochodu z nową marką i nowym modelem,
- utworzenie wypożyczenia i zmiana statusu samochodu,
- rozpoczęcie wypożyczenia,
- zakończenie wypożyczenia z utworzeniem płatności i zgłoszenia uszkodzenia,
- anulowanie rezerwacji no-show z ewentualną karą.

Transakcja zapewnia atomowość. Jeżeli jedna część operacji się nie powiedzie, całość jest wycofywana przez **ROLLBACK**. Dzięki temu baza nie pozostaje w częściowo zmienionym stanie.

23. Obsługa błędów

Backend posiada centralny mechanizm obsługi błędów.

Najważniejsze przypadki:

- błędy walidacji `zod` zwracają kod `400`,
- brak autoryzacji zwraca kod `401`,
- brak uprawnień zwraca kod `403`,
- brak zasobu zwraca kod `404`,
- konflikt danych, np. duplikat, zwraca kod `409`,
- naruszenie constraintów PostgreSQL jest mapowane na odpowiedni komunikat.

Przykłady obsługiwanych błędów bazy:

- `23505` - naruszenie unikalności,
- `23514` - naruszenie ograniczenia `CHECK`,
- `23503` - naruszenie klucza obcego,
- `23P01` - naruszenie ograniczenia wykluczającego, np. konflikt terminu wypożyczenia.

24. Instrukcja uruchomienia projektu

24.1. Uruchomienie backendu i bazy danych

Należy przejść do katalogu `server`:

```
cd server
```

Następnie uruchomić kontenery:

```
docker compose up -d --build
```

Po uruchomieniu:

- backend działa pod adresem `http://localhost:3000`,
- baza PostgreSQL działa w kontenerze `wypożyczalnia-db`,
- port bazy na komputerze hosta to `5433`.

Dane połączenia z bazą:

Parametr	Wartość
Host z poziomu Dockera	<code>postgres-db</code>
Host z poziomu komputera	<code>localhost</code>
Port hosta	<code>5433</code>
Port kontenera	<code>5432</code>

Parametr	Wartość
Baza danych	rental_db
Użytkownik	root
Hasło	mysecretpassword

24.2. Uruchomienie frontendu

Należy przejść do katalogu `client`:

```
cd client
```

Zainstalować zależności:

```
npm install
```

Uruchomić frontend:

```
npm run dev
```

Frontend komunikuje się z backendem pod adresem:

```
http://localhost:3000
```

24.3. Konto testowe administratora

W bazie seedowane jest konto testowe:

```
Email: admin@admin.com  
Hasło: adminadmin  
Rola: Admin
```

25. Podsumowanie

Projekt realizuje kompletny system wypożyczalni samochodów z relacyjną bazą danych PostgreSQL. Model danych obejmuje najważniejsze elementy domeny: użytkowników, oddziały, marki, modele, samochody, wypożyczenia, płatności, serwisy i uszkodzenia.

Najważniejsze aspekty techniczne projektu to:

- zastosowanie znormalizowanego modelu relacyjnego,
- wykorzystanie kluczy głównych i obcych,
- walidacja danych na poziomie backendu i bazy,
- wykorzystanie typów `ENUM`,
- ograniczenia `CHECK` i `UNIQUE`,
- widoki raportowe,
- indeksy poprawiające wydajność,
- transakcje bazodanowe dla złożonych operacji,
- ograniczenie `EXCLUDE` blokujące nakładające się rezerwacje,
- REST API z autoryzacją opartą o JWT,
- hashowanie haseł użytkowników.

Baza danych nie pełni wyłącznie roli magazynu danych. Została wykorzystana również jako warstwa kontroli spójności, raportowania i zabezpieczania reguł biznesowych. Dzięki temu system jest bardziej odporny na błędy aplikacji i zachowuje integralność danych nawet przy bardziej złożonych operacjach.